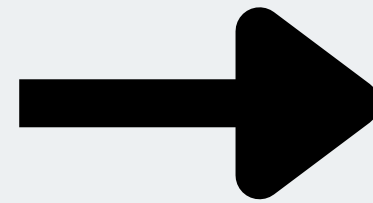

NEURAL ORDINARY DIFFERENTIAL EQUATIONS

Corduneanu-Huci Mia
Haltrich Thanika

NEURAL ORDINARY DIFFERENTIAL EQUATIONS

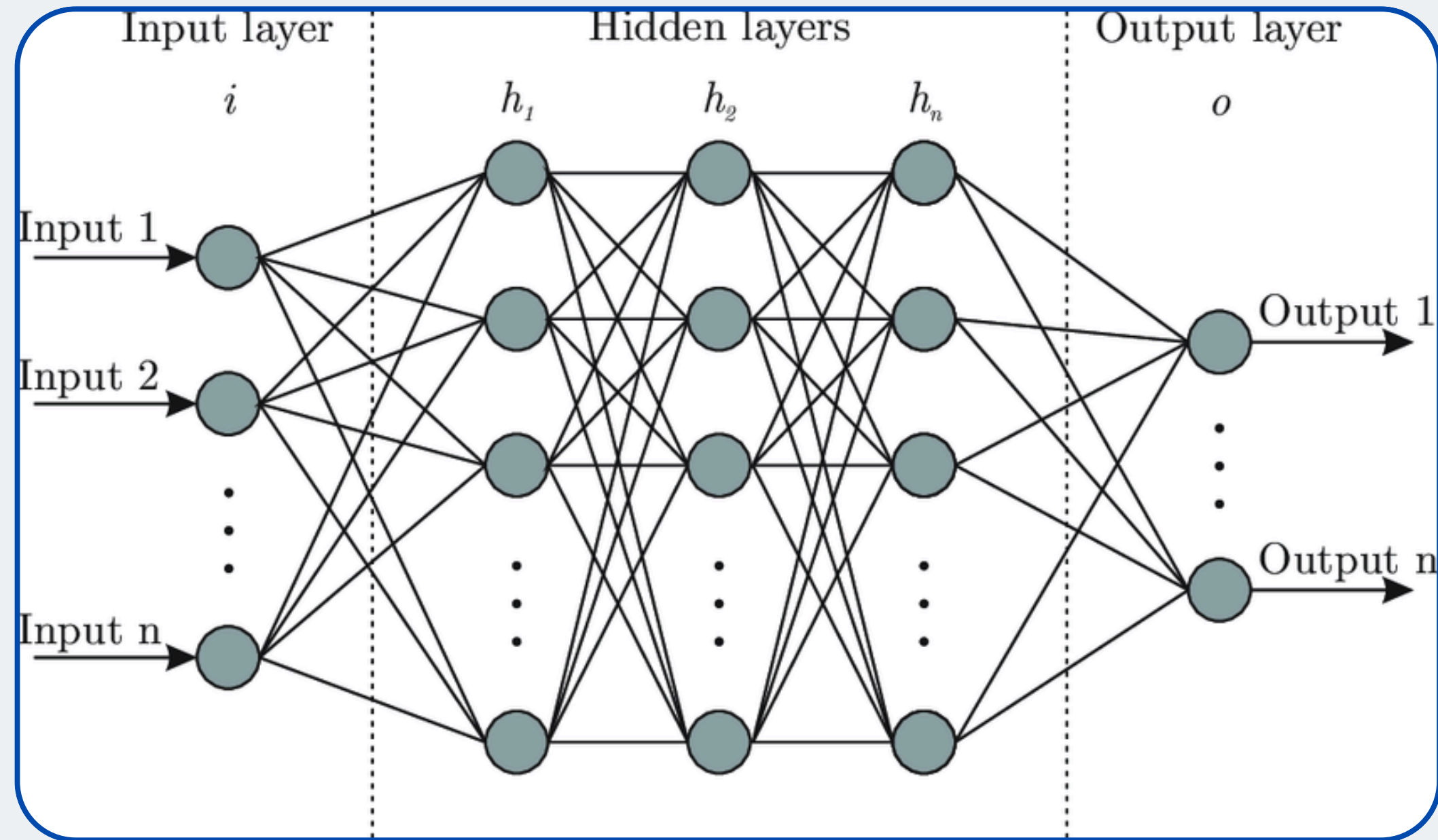
a sequence of discrete layers



continuous dynamical system

STANDARD NEURAL NETWORKS

STRUCTURE



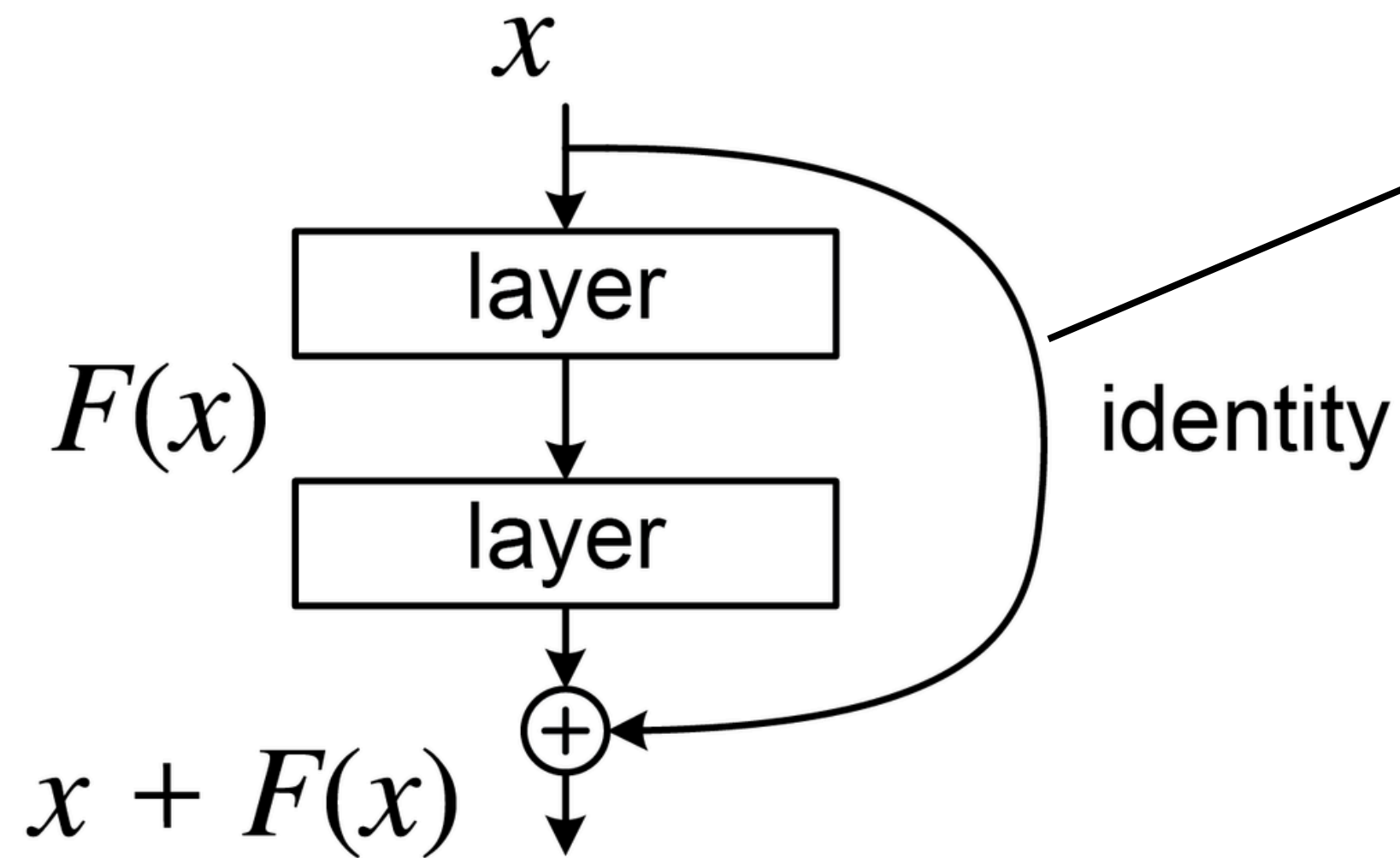
next hidden state current hidden state

$$h_{t+1} = \sigma(W_t h_t + b_t)$$

activation function

RESIDUAL NEURAL NETWORK

STRUCTURE



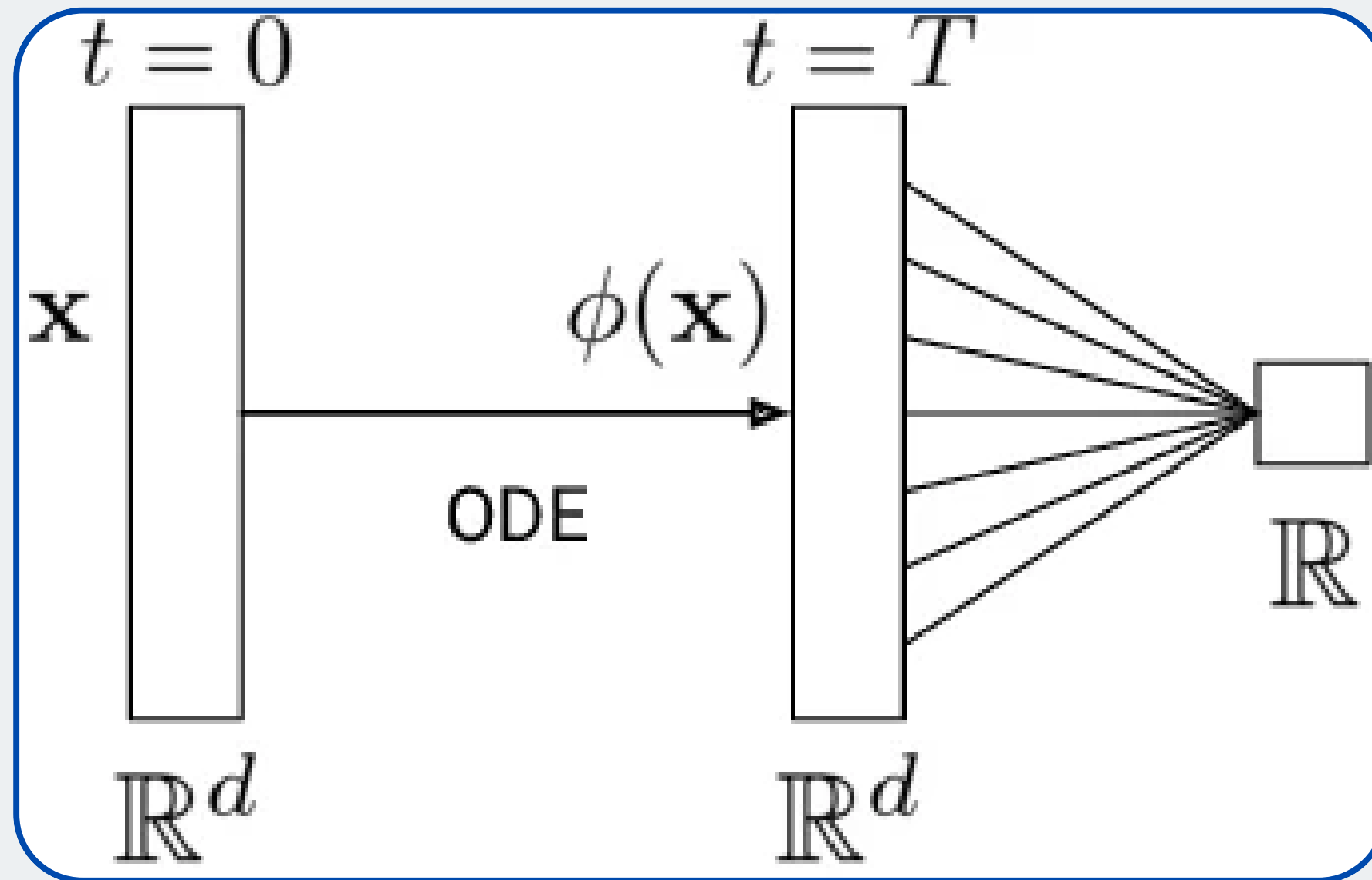
skip connection

$$h_{t+1} = h_t + f(h_t, \theta_t)$$

- a discrete approximation of a continuous process

NEURAL ORDINARY DIFFERENTIAL EQUATION

STRUCTURE

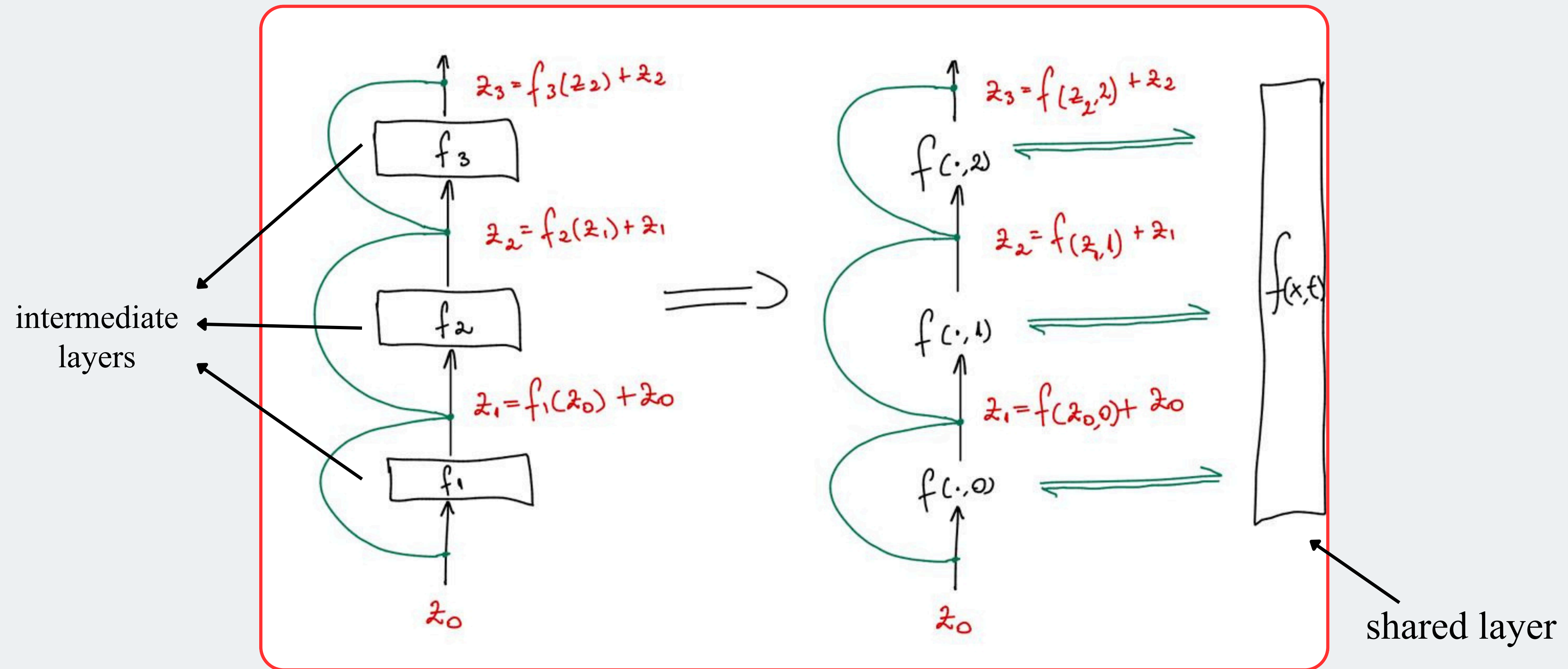


How fast is h changing at t ?

$$\frac{dh(t)}{dt} = f_{\theta}(h(t), t)$$

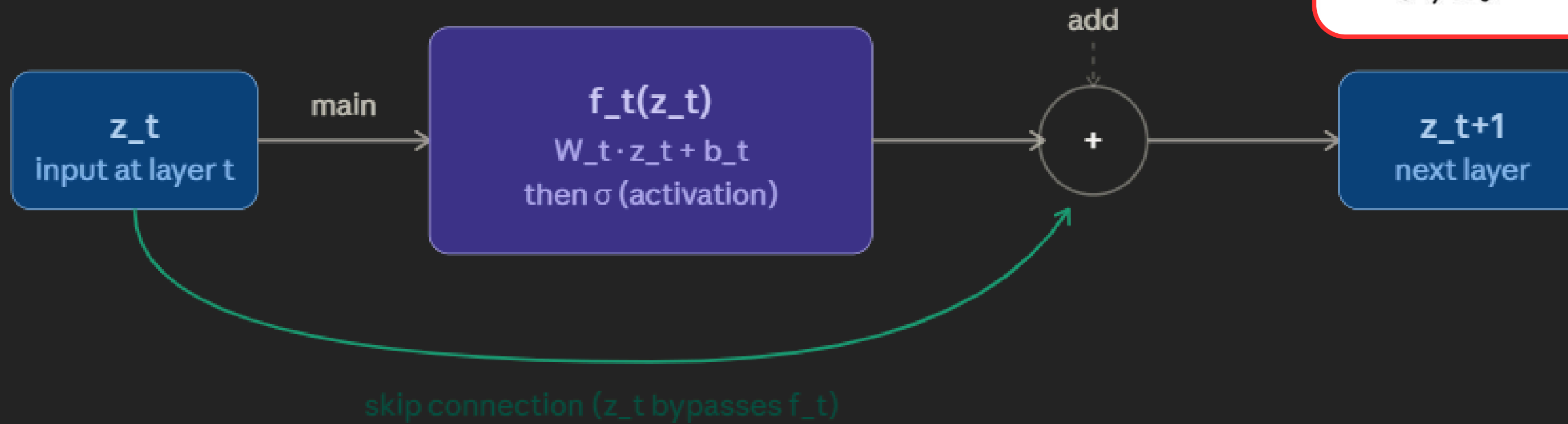
- instead of jumping between layers, h flows continuously from $t=0$ to $t=T$

RESNET VS ODENET



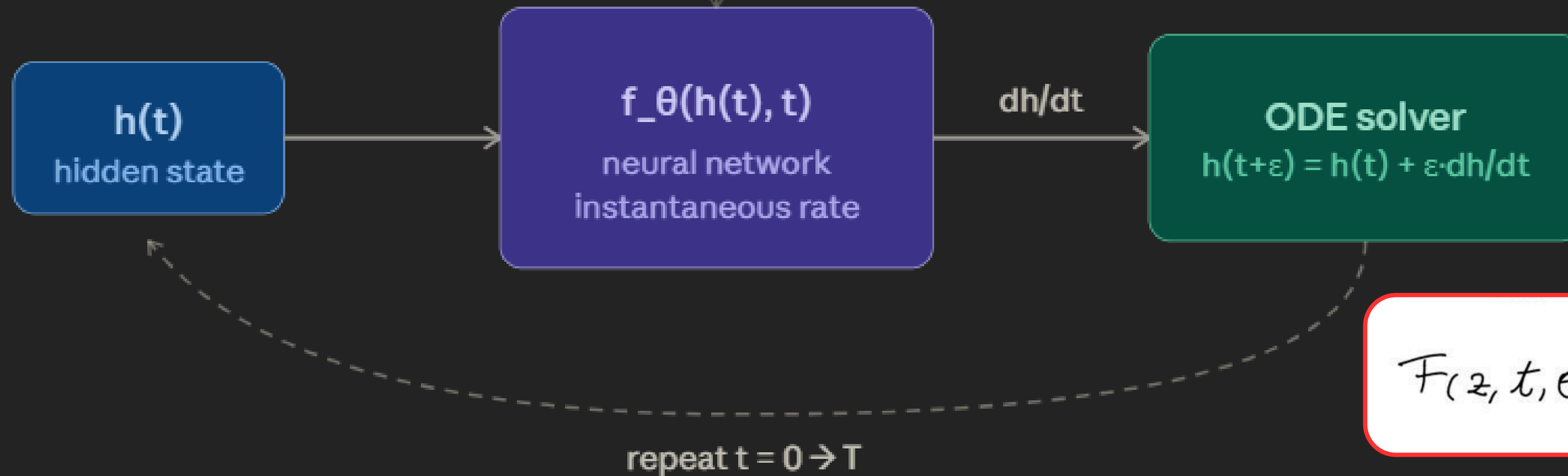
residual connection: output = transformation + original input

$$F(z, \theta) \text{ or } F(f_1, f_2, f_3, z, \theta)$$



t (depth/time)

final representation
 $\rightarrow \varphi(x) = h(T)$

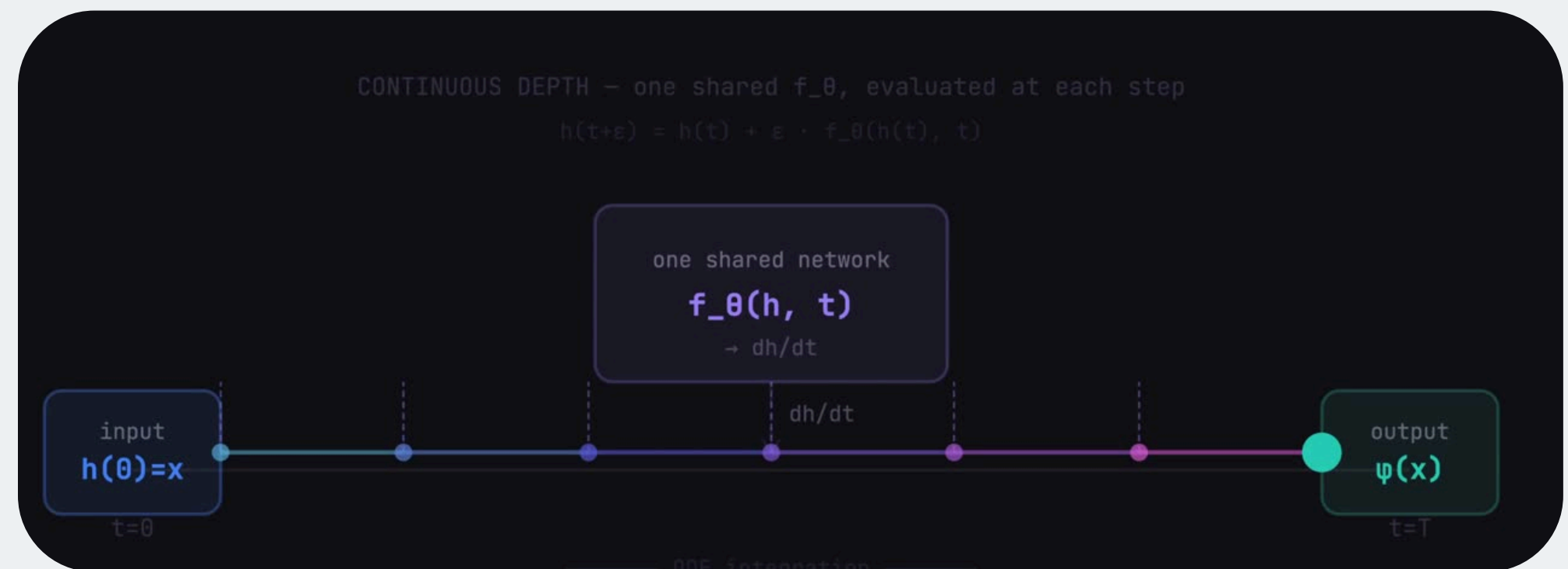


$$F(z, t, \theta) \text{ or } F(f, z, t, \theta)$$

NEURAL ORDINARY DIFFERENTIAL EQUATION

STEPS:

1. INPUT
2. FORWARD PASS
3. BACKPROPAGATION
4. OUTPUT



INPUT

$h(t_0) = x$ → x mapped to the initial hidden state

- input data interpreted as initial condition
- starting point of a continuous trajectory

FORWARD PASS

$$\frac{dh(t)}{dt} = f_{\theta}(h(t), t)$$



neural network

~~sequence of layers~~

solving a differential equation

$$h(t_1) = h(t_0) + \int_{t_0}^{t_1} f_{\theta}(h(t), t) dt$$

approximate



$$h(t_1) = \text{ODESolve}(f_{\theta}, h(t_0), t_0, t_1)$$

- sequence of intermediate states that approximate a continuous trajectory of the system

BACKPROPAGATION

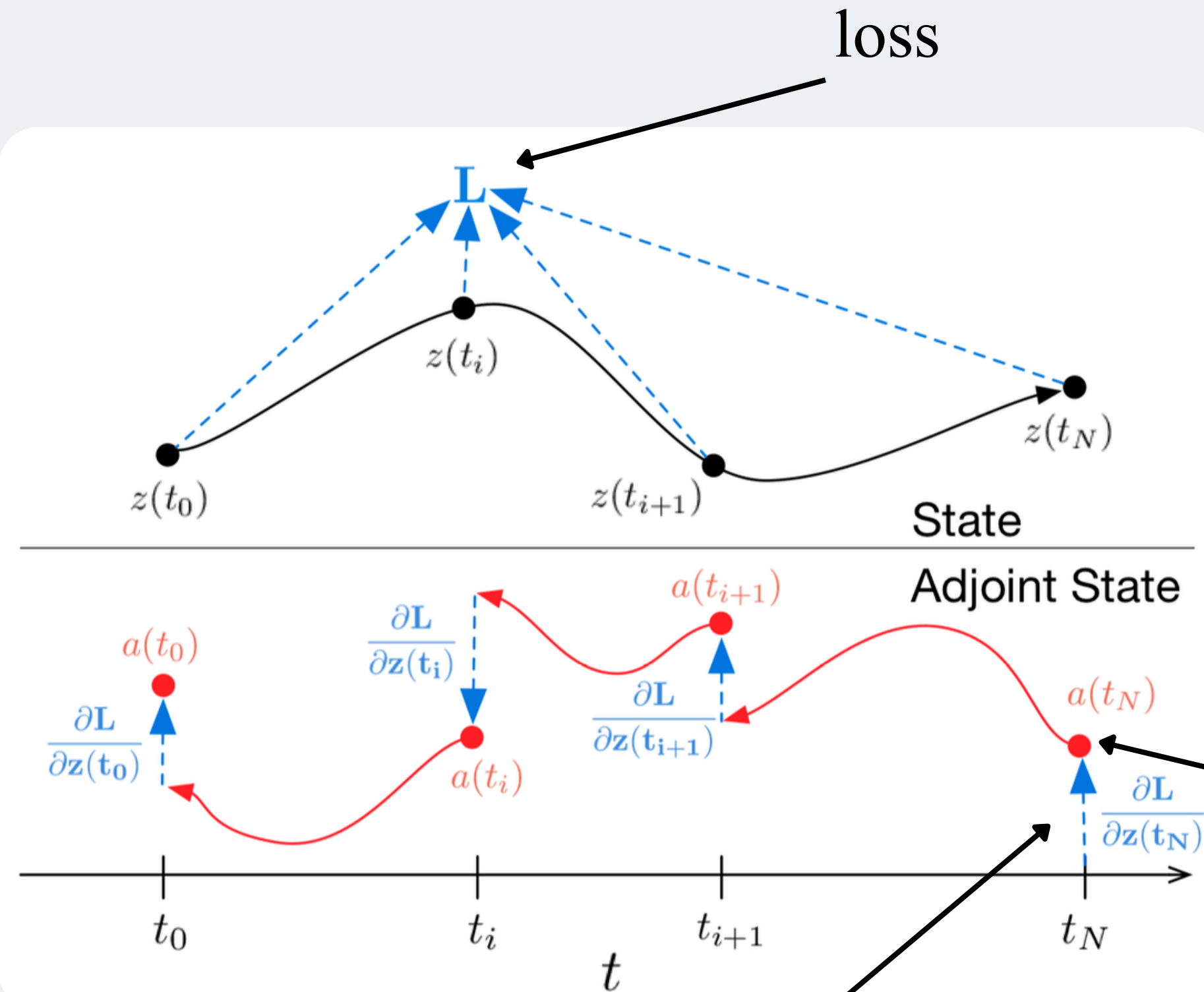
~~store all intermediate states during forward pass
+
backpropagation~~

ADJOINT SENSITIVITY METHOD

$$a(t) = \frac{dL}{dh(t)}$$

- solved backward in time
- from t_1 to t_0

ADJOINT STATE



$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f_{\theta}(h(t), t)}{\partial h(t)}$$

$$\frac{dL}{d\theta} = - \int_{t_1}^{t_0} a(t)^T \frac{\partial f_{\theta}}{\partial \theta} dt$$

how sensitive the loss is to the hidden state at each moment

gradient of the loss with respect to z

OUTPUT

$h(t_1)$

hidden state at the final time

- transformed input after continuous evolution through the learned dynamics
- hidden state defined for all t in $[t_0, t_1]$

prediction final time + intermediate points

Optimal control theory: system ➡ control behaviour ➡ minimize objective

Neural Ordinary Differential Equations

Ricky T. Q. Chen*, Yulia Rubanova*, Jesse Bettencourt*, David Duvenaud
University of Toronto, Vector Institute
{rtqichen, rubanova, jessebett, duvenaud}@cs.toronto.edu

Abstract

We introduce a new family of deep neural network models. Instead of specifying a discrete sequence of hidden layers, we parameterize the derivative of the hidden state using a neural network. The output of the network is computed using a black-box differential equation solver. These continuous-depth models have constant memory cost, adapt their evaluation strategy to each input, and can explicitly trade numerical precision for speed. We demonstrate these properties in continuous-depth residual networks and continuous-time latent variable models. We also construct continuous normalizing flows, a generative model that can train by maximum likelihood, without partitioning or ordering the data dimensions. For training, we show how to scalably backpropagate through any ODE solver, without access to its internal operations. This allows end-to-end training of ODEs within larger models.

1 Introduction

Models such as residual networks, recurrent neural network decoders, and normalizing flows build complicated transformations by composing a sequence of transformations to a hidden state:

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t) \quad (1)$$

where $t \in \{0 \dots T\}$ and $\mathbf{h}_t \in \mathbb{R}^D$. These iterative updates can be seen as an Euler discretization of a continuous transformation (Lu et al., 2017; Haber and Ruthotto, 2017; Ruthotto and Haber, 2018).

What happens as we add more layers and take smaller steps? In the limit, we parameterize the continuous dynamics of hidden units using an ordinary differential equation (ODE) specified by a neural network:

$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta) \quad (2)$$

Starting from the input layer $\mathbf{h}(0)$, we can define the output layer $\mathbf{h}(T)$ to be the solution to this ODE initial value problem at some time T . This value can be computed by a black-box differential equation solver, which evaluates the hidden unit dynamics f wherever necessary to determine the solution with the desired accuracy. Figure 1 contrasts these two approaches.

Defining and evaluating models using ODE solvers has several benefits:

Memory efficiency In Section 2, we show how to compute gradients of a scalar-valued loss with respect to all inputs of any ODE solver, *without backpropagating through the operations of the solver*. Not storing any intermediate quantities of the forward pass allows us to train our models with constant memory cost as a function of depth, a major bottleneck of training deep models.

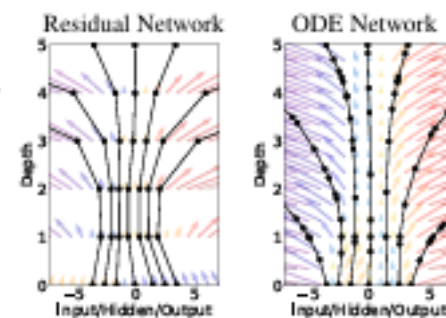


Figure 1: *Left:* A Residual network defines a discrete sequence of finite transformations. *Right:* A ODE network defines a vector field, which continuously transforms the state. *Both:* Circles represent evaluation locations.

CLASSICS OF SOVIET MATHEMATICS

L. S. PONTRYAGIN SELECTED WORKS

IN FOUR VOLUMES

VOLUME FOUR

The Mathematical Theory of Optimal Processes

CRC Press
Taylor & Francis Group

hidden state

system evolving
over time

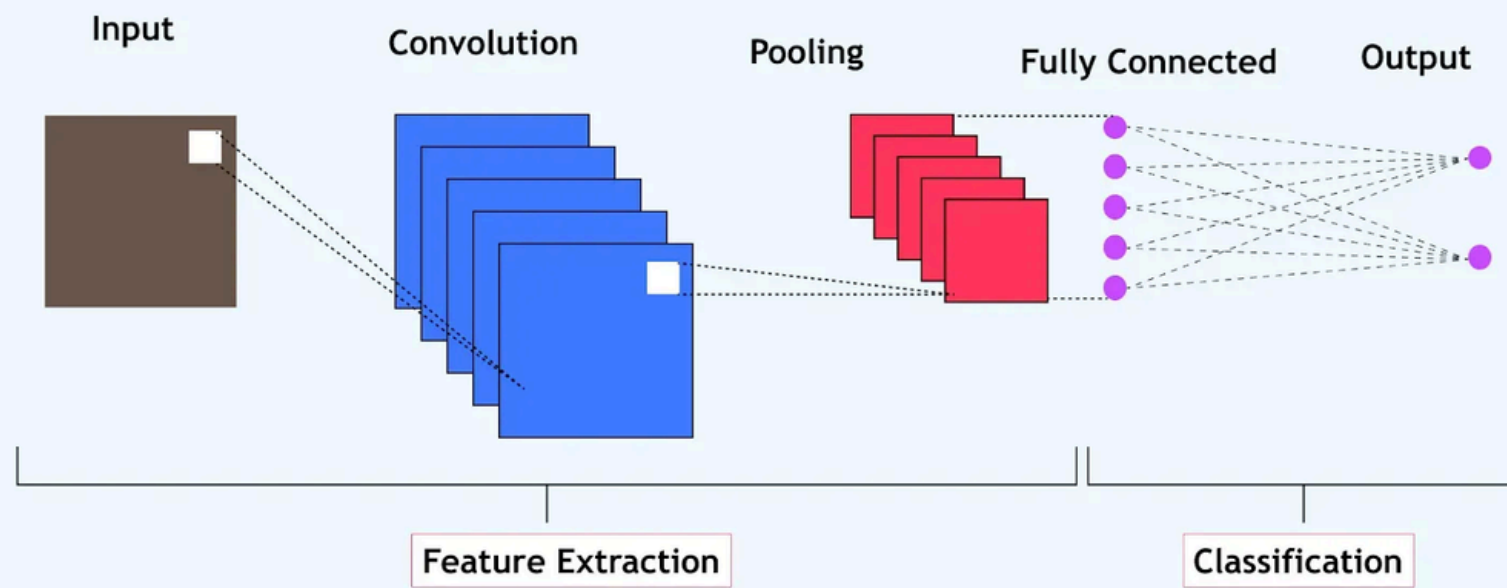
neural network

controls how the
system moves

CODE IMPLEMENTATION

CODING A CONTINUOUS LAYER???

The Architecture of Convolutional Neural Networks



```
self.conv_1 = torch.nn.Conv2d(in_channels=1,...)

self.pool_1 = torch.nn.MaxPool2d(kernel_size=(2, 2),...)

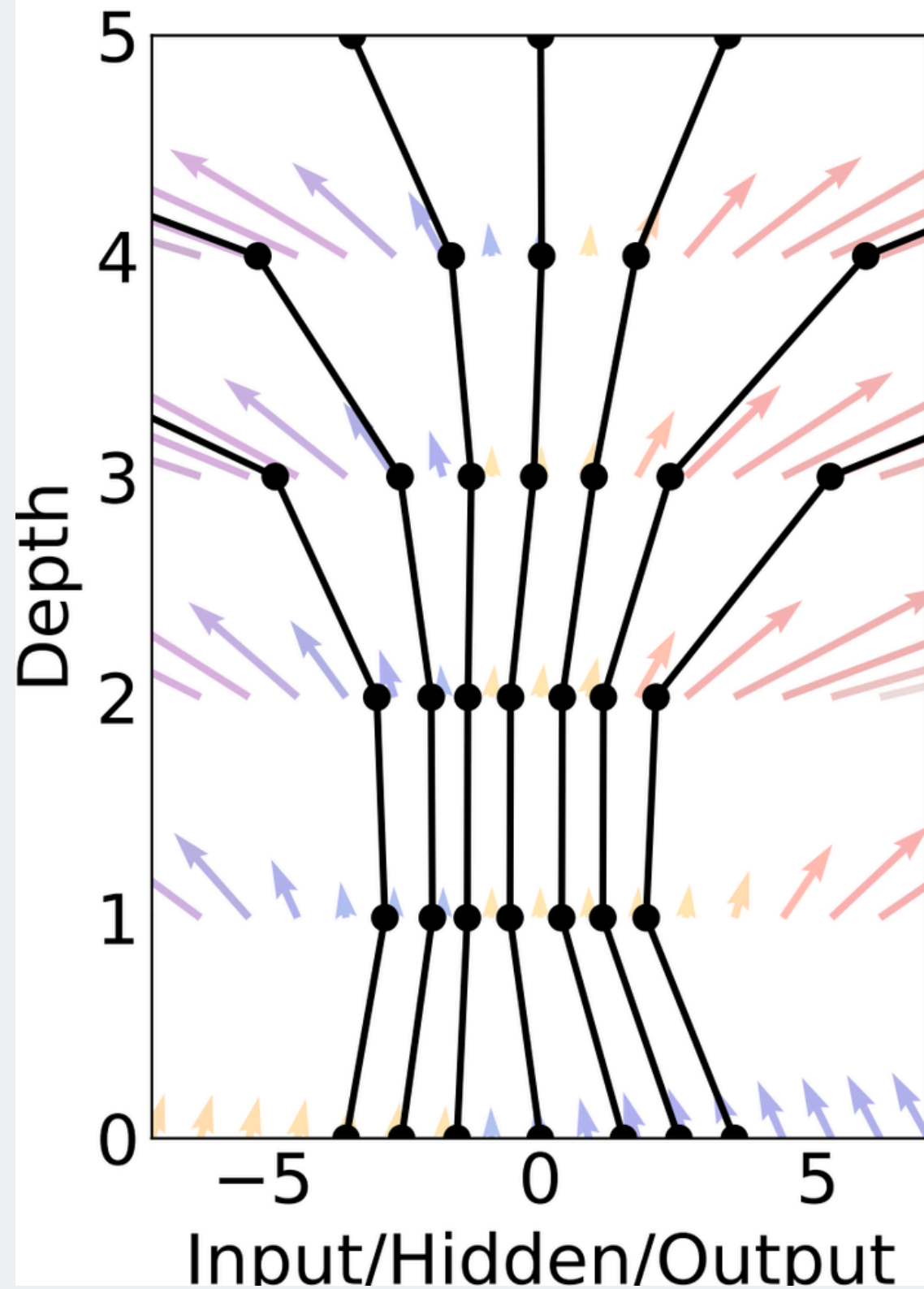
self.conv_2 = torch.nn.Conv2d(in_channels=8,...)
```

library:torchdiffeq

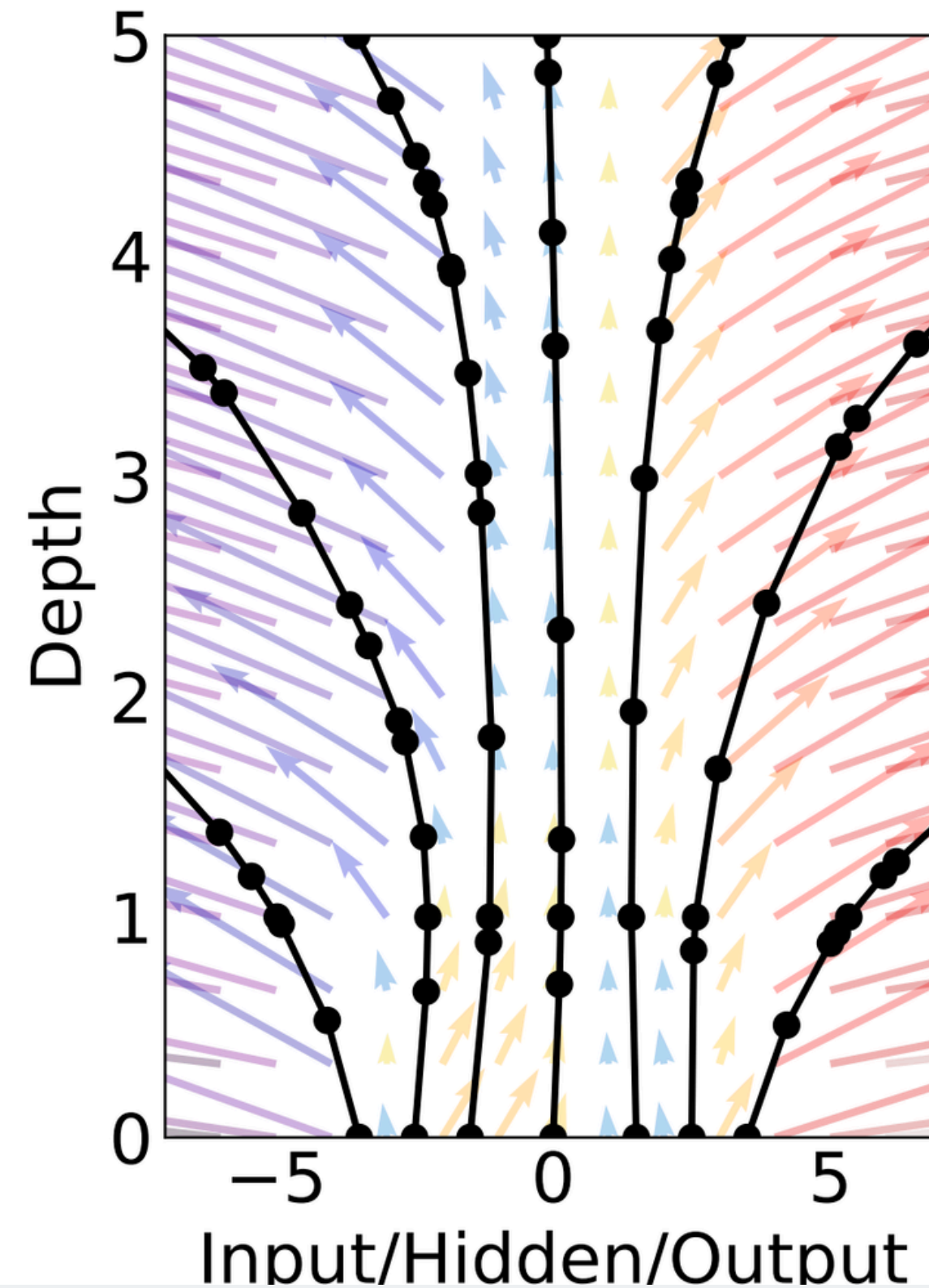
CODE IMPLEMENTATION

WHAT EVEN IS A CONTINUOUS LAYER?

Residual Network



ODE Network



CODE IMPLEMENTATION

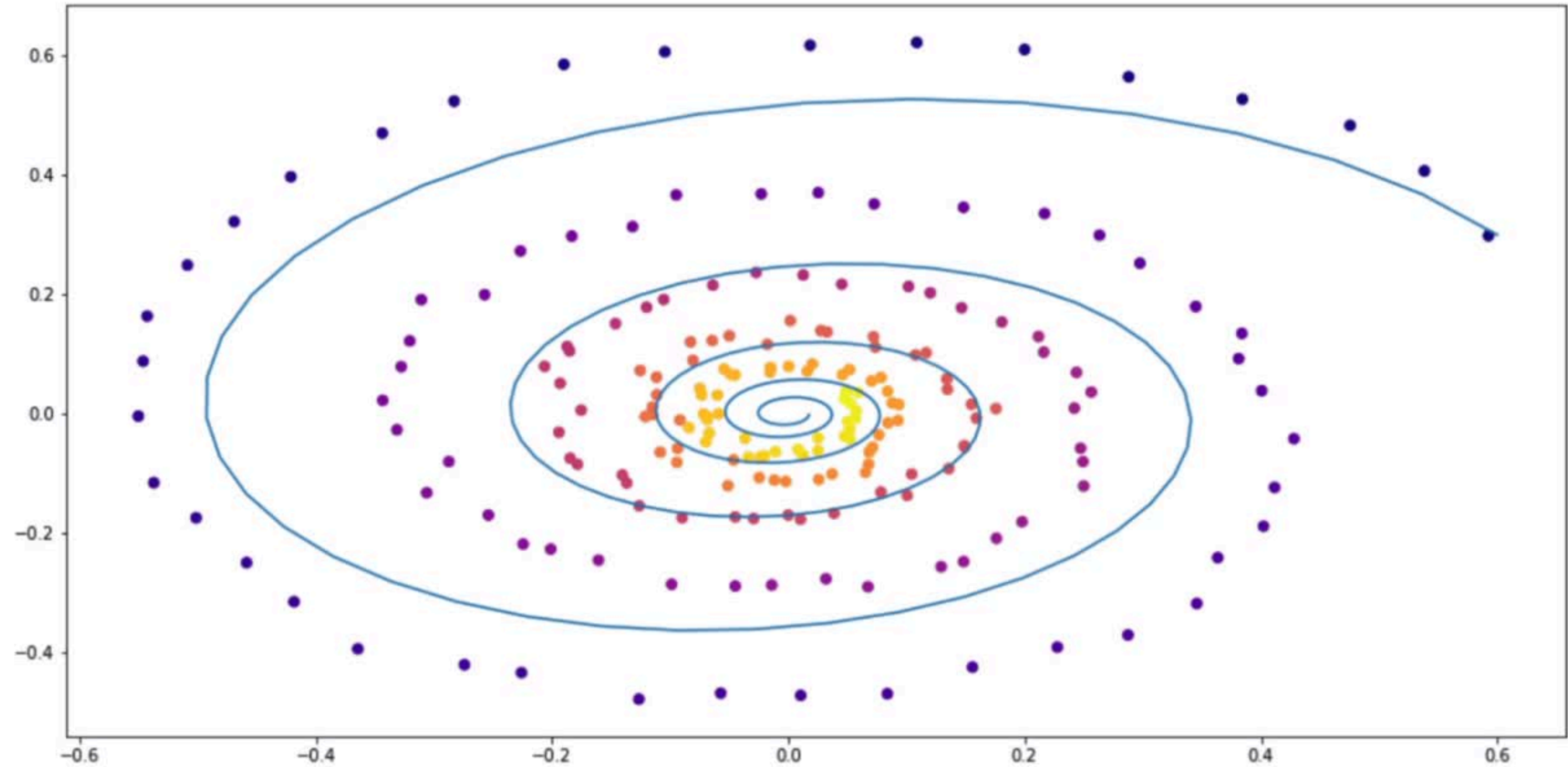
WHAT EVEN IS A CONTINUOUS LAYER?

```
class ODEAdjoint(torch.autograd.Function):
    @staticmethod
    def forward(ctx, z0, t, flat_parameters, func):
        '''calling the ode solver for each time step'''
        assert isinstance(func, ODEF)
        bs, *z_shape = z0.size()
        time_len = t.size(0)

        with torch.no_grad(): #we don't save gradients
            z = torch.zeros(time_len, bs, *z_shape).to(z0)
            z[0] = z0
            for i_t in range(time_len - 1): # --(1)
                z0 = ode_solve(z0, t[i_t], t[i_t+1], func)
                z[i_t+1] = z0

        ctx.func = func
        ctx.save_for_backward(t, z.clone(), flat_parameters)
        #ctx is what is saved and will be used for backprop later
        return z
```

PROOF OF CONCEPT



PROOF OF CONCEPT

Z0 INITIAL VAULE

OTHER VALUES AT
RANDOM TIMES

```
def conduct_experiment(ode_true, ode_trained, n_steps, name, plot_freq=10):
    # Create data
    z0 = Variable(torch.Tensor([[0.6, 0.3]]))

    # Get trajectory of random timespan
    min_delta_time = 1.0
    max_delta_time = 5.0
    max_points_num = 32
    def create_batch():
        t0 = np.random.uniform(0, t_max - max_delta_time)
        t1 = t0 + np.random.uniform(min_delta_time, max_delta_time)

        idx = sorted(np.random.permutation(index_np[(times_np > t0) & (times_np < t1)])[:max_points_num])

        obs_ = obs[idx]
        ts_ = times[idx]
        return obs_, ts_

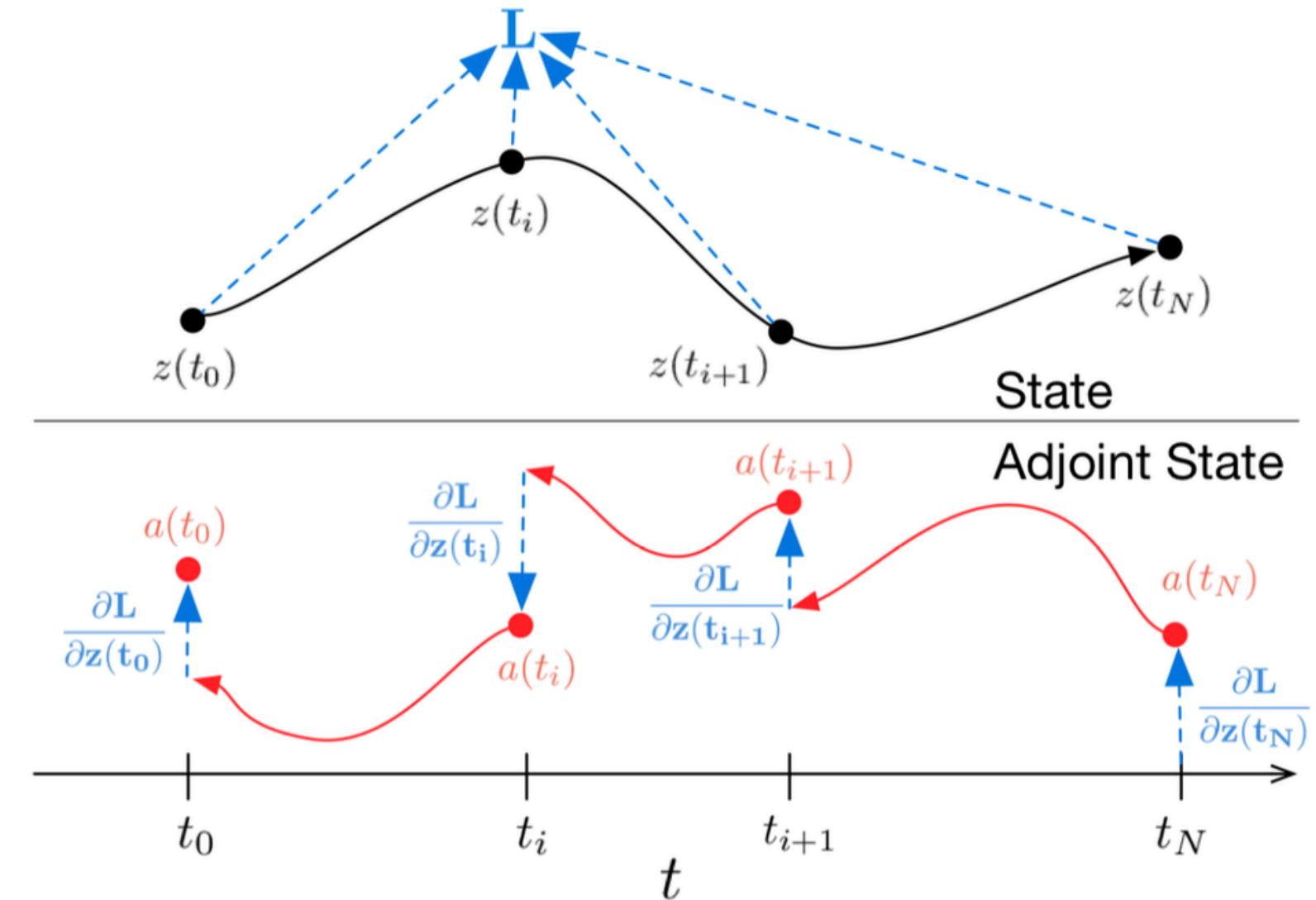
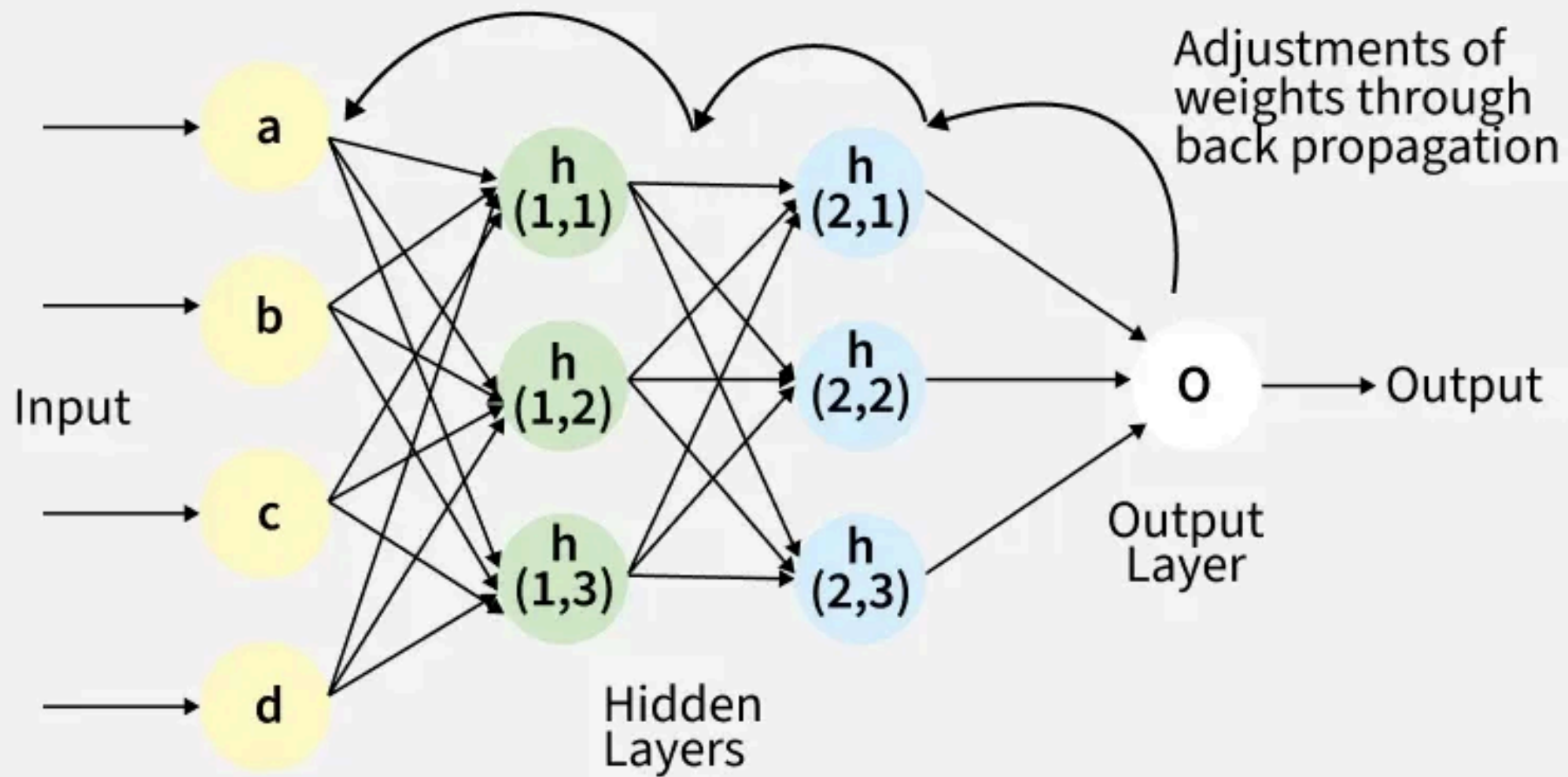
    # Train Neural ODE
    optimizer = torch.optim.Adam(ode_trained.parameters(), lr=0.01)
    for i in range(n_steps):
        obs_, ts_ = create_batch()

        z_ = ode_trained(obs_[0], ts_, return_whole_sequence=True)
        loss = F.mse_loss(z_, obs_.detach())

        optimizer.zero_grad()
        loss.backward(retain_graph=True)
        optimizer.step()
```

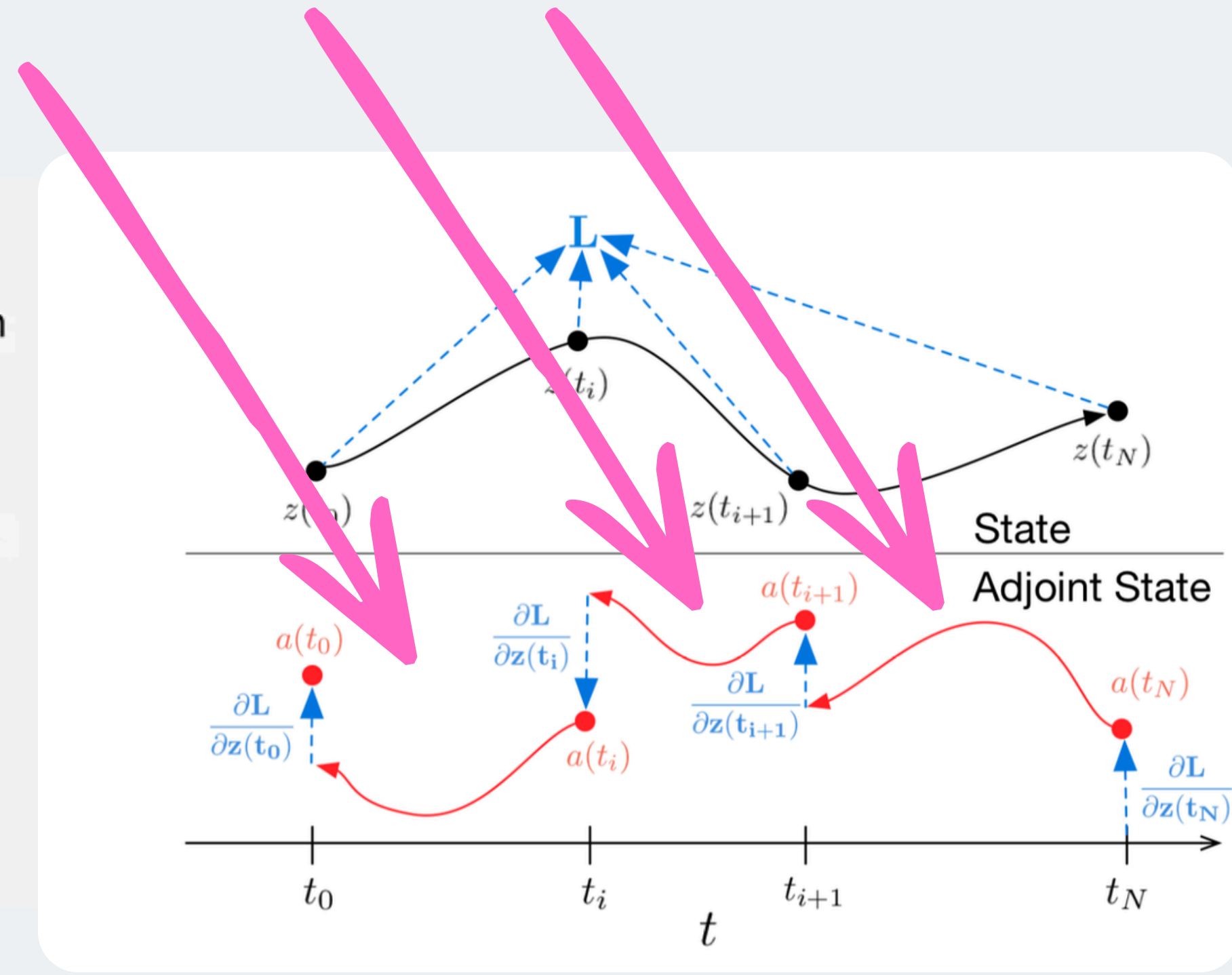
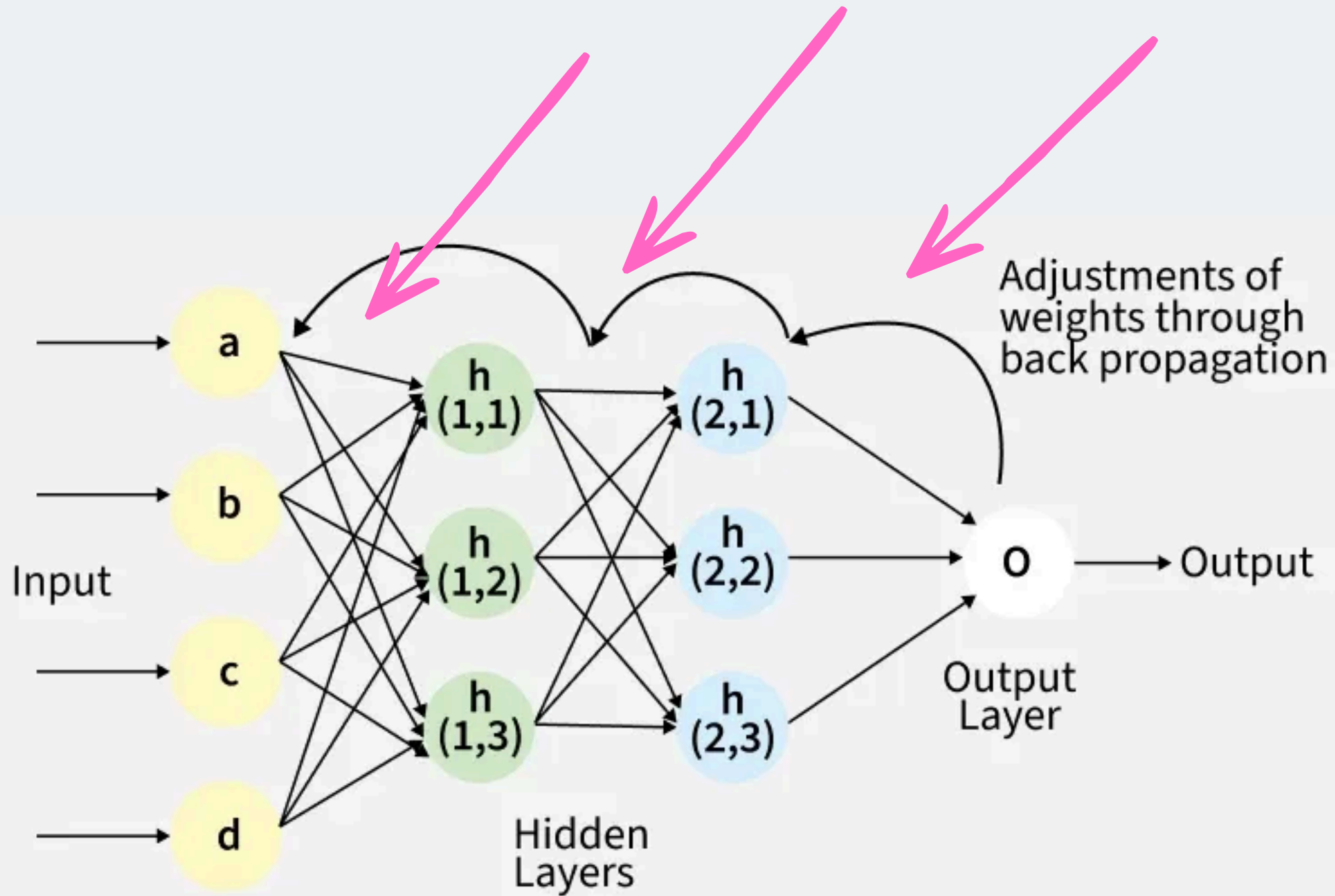
CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE



CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE

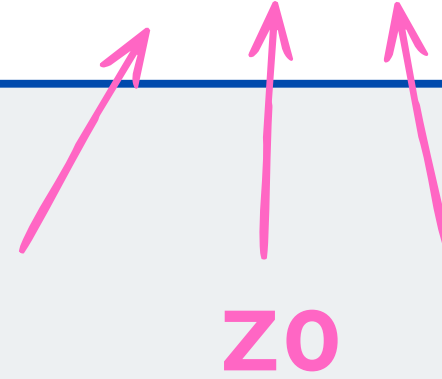


CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE

The output of the model is the hidden state at the final time t_1 , which is obtained by numerically integrating the differential equation:

$$h(t_1) = \text{ODESolve}(f_\theta, h(t_0), t_0, t_1)$$

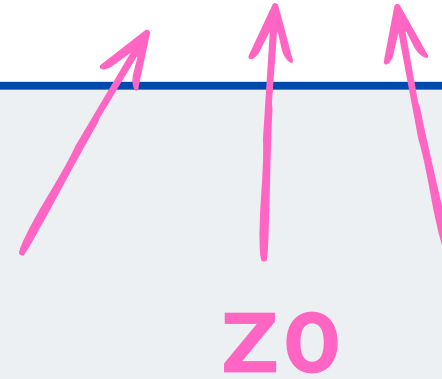


CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE

The output of the model is the hidden state at the final time t_1 , which is obtained by numerically integrating the differential equation:

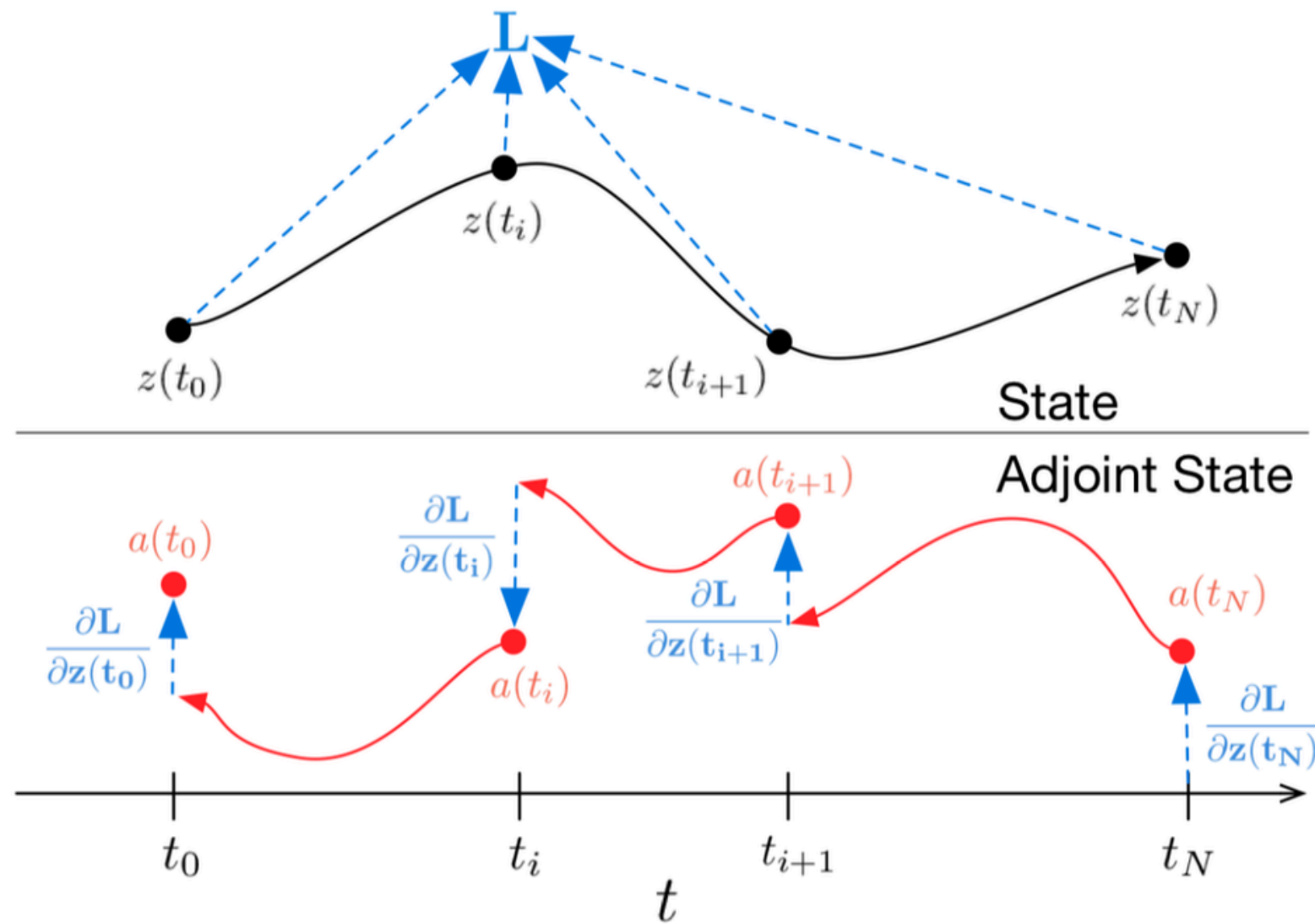
$$h(t_1) = \text{ODESolve}(f_\theta, h(t_0), t_0, t_1)$$



$$\frac{\partial L}{\partial z(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}$$

CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE



$$\frac{da(t)}{dt} = -a(t) \frac{\partial f(z(t), t, \theta)}{\partial z}$$

SOPHISTICATED MATH

$$\frac{da_{\text{aug}}}{dt} = - \begin{bmatrix} a \frac{\partial f}{\partial z} & a \frac{\partial f}{\partial \theta} & a \frac{\partial f}{\partial t} \end{bmatrix}$$

CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE

$$\frac{da_{\text{aug}}}{dt} = - \begin{bmatrix} a \frac{\partial f}{\partial z} & a \frac{\partial f}{\partial \theta} & a \frac{\partial f}{\partial t} \end{bmatrix}$$

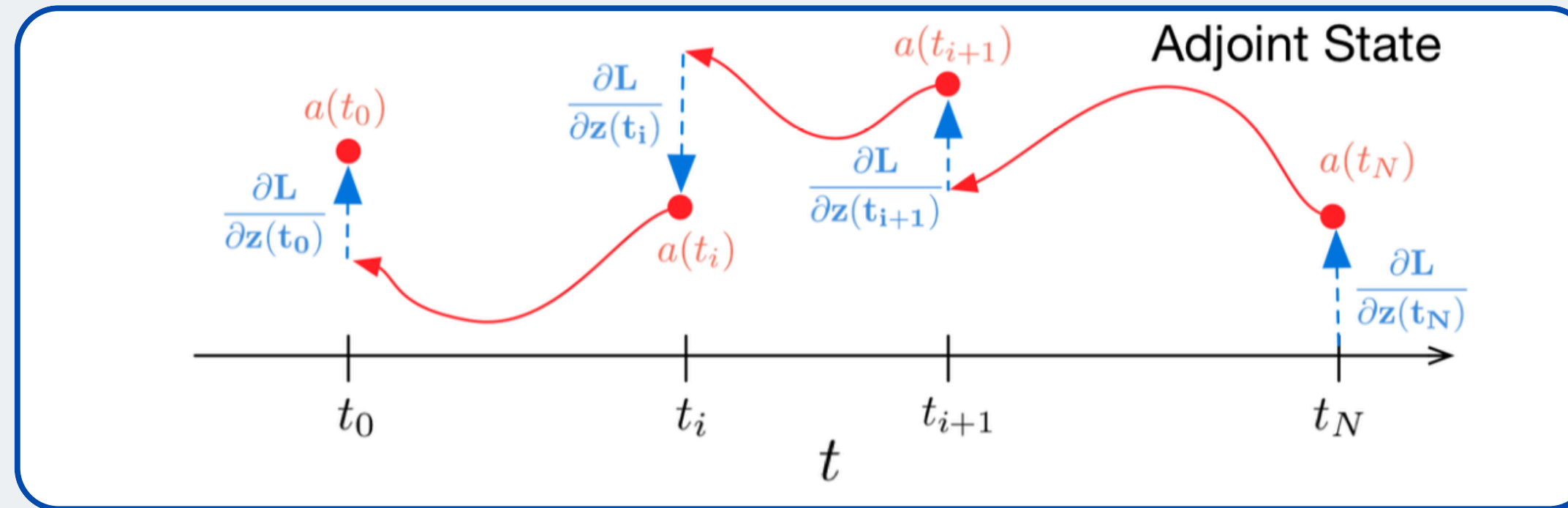
```
def forward_with_grad(self, z, t, grad_outputs):
    """Compute f and a df/dz, a df/dp, a df/dt"""
    batch_size = z.shape[0]

    out = self.forward(z, t)

    a = grad_outputs
    adfdz, adfdt, *adfdp = torch.autograd.grad( # --(1)
        # gradients needed to find da_aug/dt using the adjoint state
        (out,), (z, t) + tuple(self.parameters()), grad_outputs=(a),
        allow_unused=True, retain_graph=True
    )
```

CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE



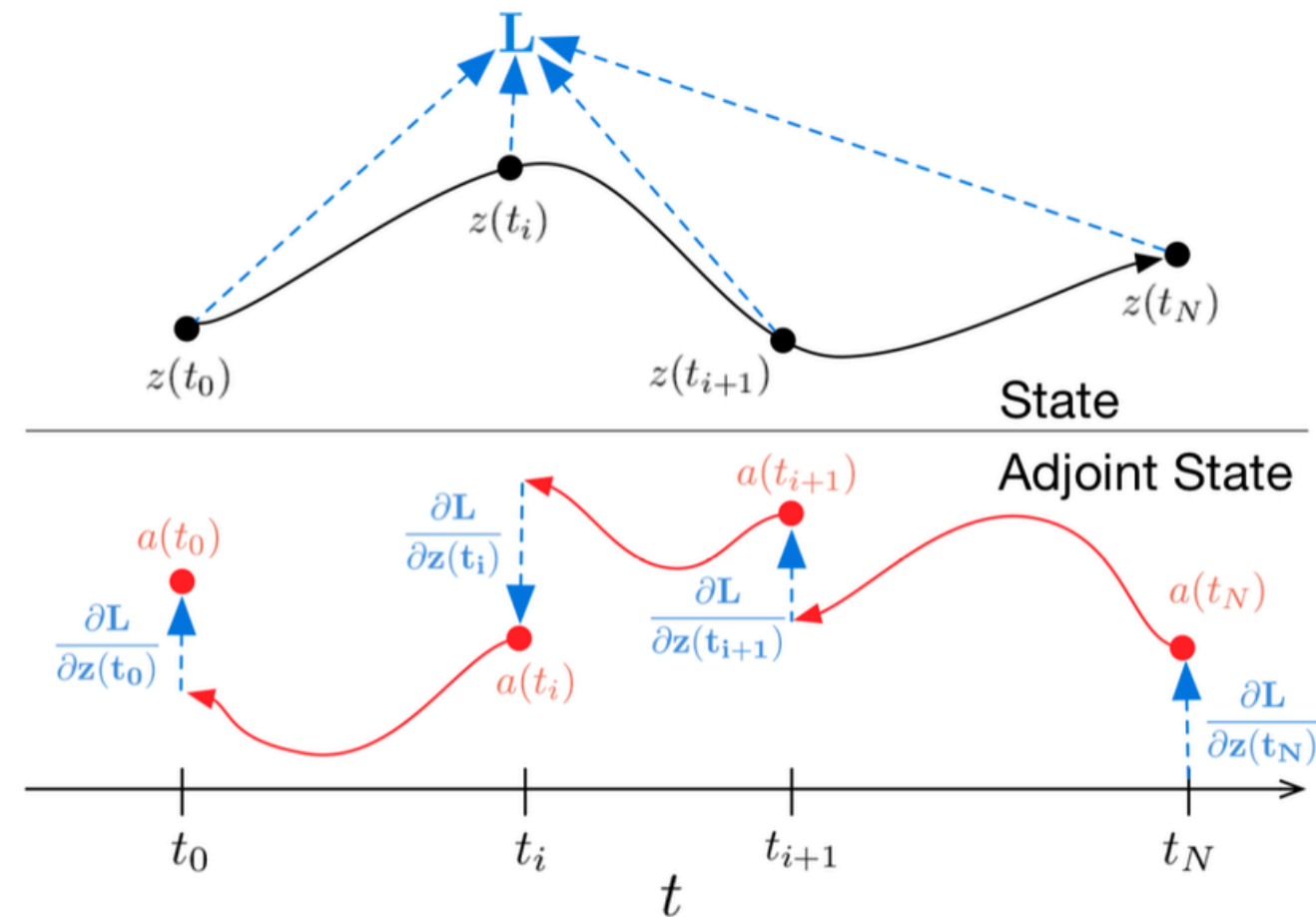
```
# Unflatten z and a
z_i = z_i.view(bs, *z_shape)
a = a.view(bs, *z_shape)
with torch.set_grad_enabled(True): # we are tracking gradients again since we will need it for backprop
    t_i = t_i.detach().requires_grad_(True)
    z_i = z_i.detach().requires_grad_(True)
    # this line is when we are calculating gradients:
    func_eval, adfdz, adfdt, adfdp = func.forward_with_grad(z_i, t_i, grad_outputs=a) # bs, *z_shape
    adfdz = adfdz.to(z_i) if adfdz is not None else torch.zeros(bs, *z_shape).to(z_i)
    adfdp = adfdp.to(z_i) if adfdp is not None else torch.zeros(bs, n_params).to(z_i)
    adfdt = adfdt.to(z_i) if adfdt is not None else torch.zeros(bs, 1).to(z_i)
```

CODE IMPLEMENTATION

BACKPROPAGATION WITH THE ADJOINT STATE

$$\frac{\partial L}{\partial z(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}$$

```
return adj_z.view(bs, *z_shape), adj_t, adj_p, None
```



ACCURACY, MEMORY, TIME COMPLEXITY

	Test Error	# Params	Memory	Time
1-Layer MLP [†]	1.60%	0.24 M	-	-
ResNet	0.41%	0.60 M	$\mathcal{O}(L)$	$\mathcal{O}(L)$
ODE-Net	0.42%	0.22 M	$\mathcal{O}(1)$	$\mathcal{O}(\tilde{L})$

COMPARISION OF MODELS

MNIST DATA

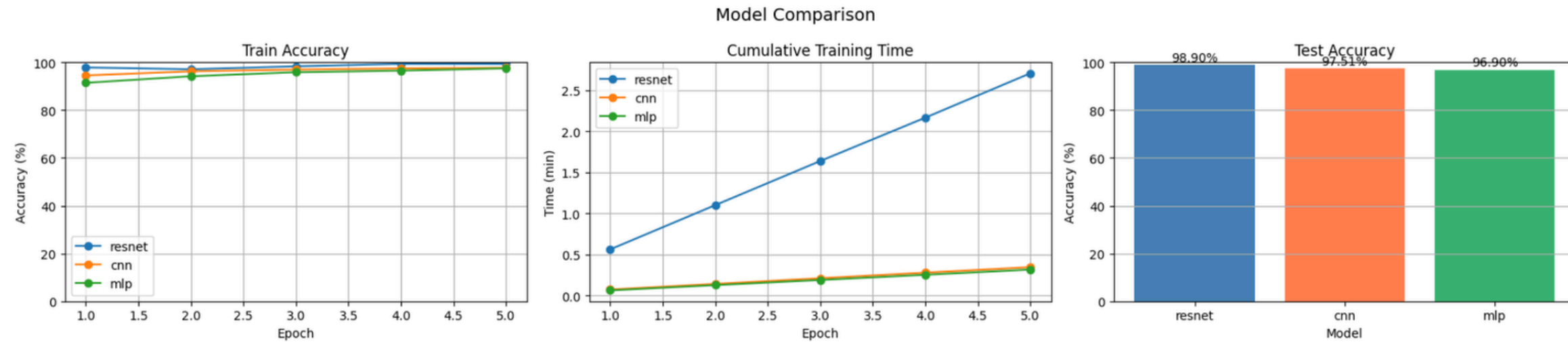


Figure 2: Comparision of Resnet, CNN, and MLP on MNIST dataset

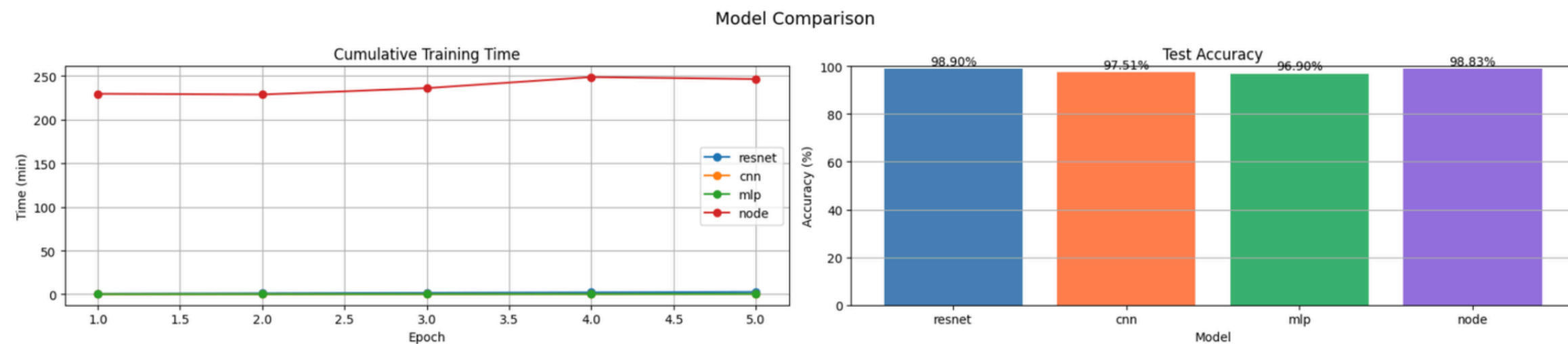


Figure 3: Comparision of Resnet, CNN, MLP, and N-ODE (using 2-order Runge-Kutta Solver) on MNIST dataset

COMPARISION OF MODELS

CIFAR10 DATA

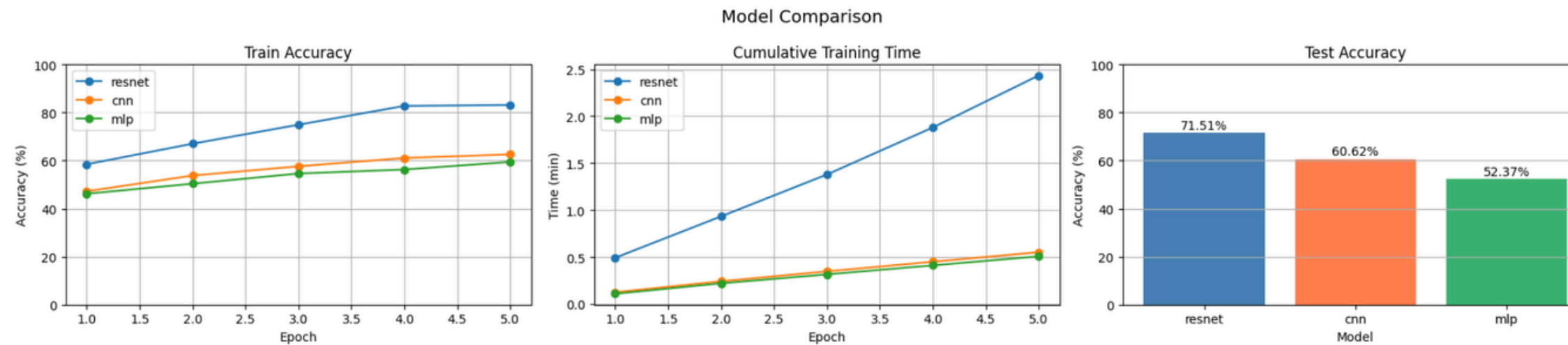


Figure 4: Comparision of Resnet, CNN, and MLP on CIFAR10 dataset

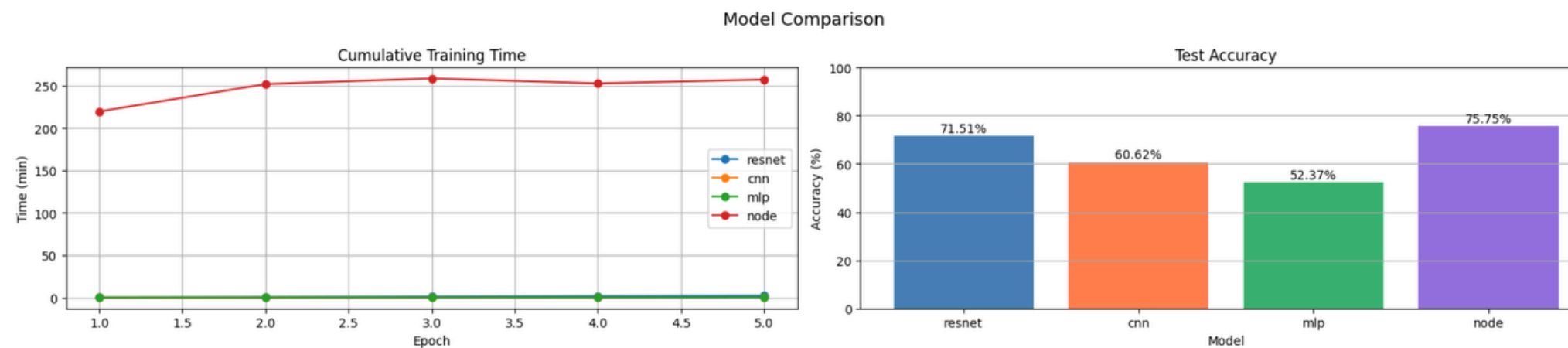
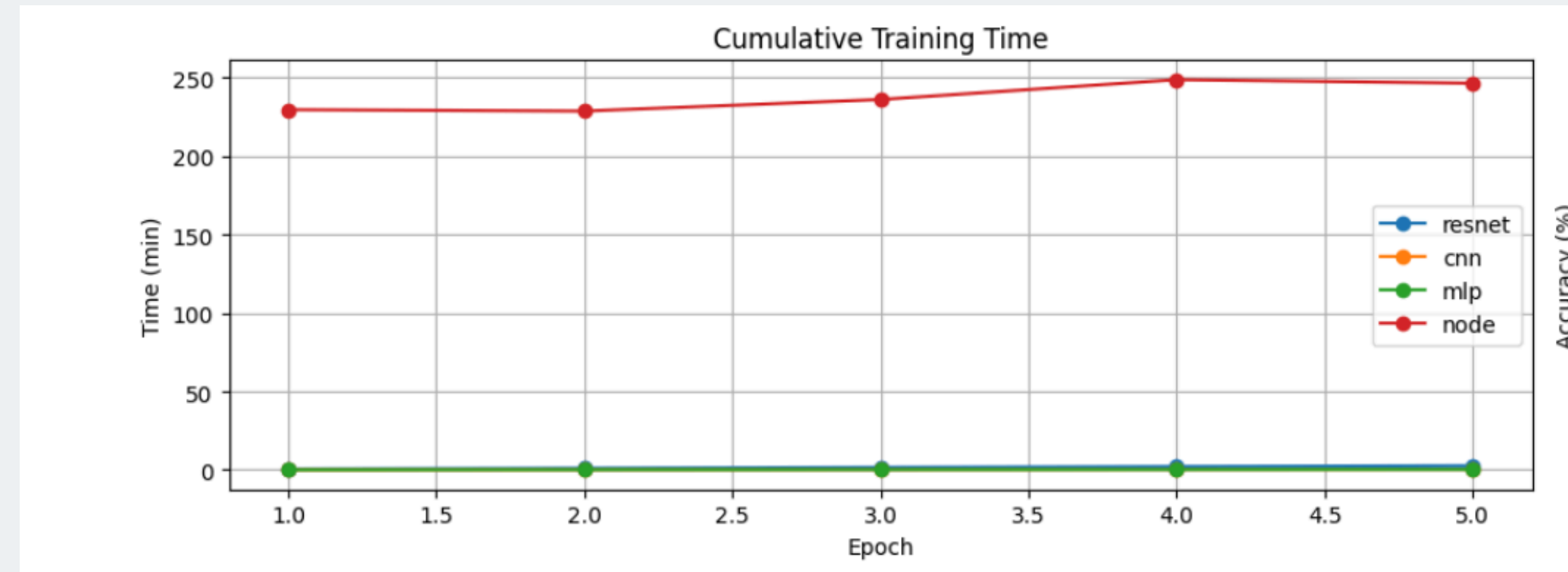


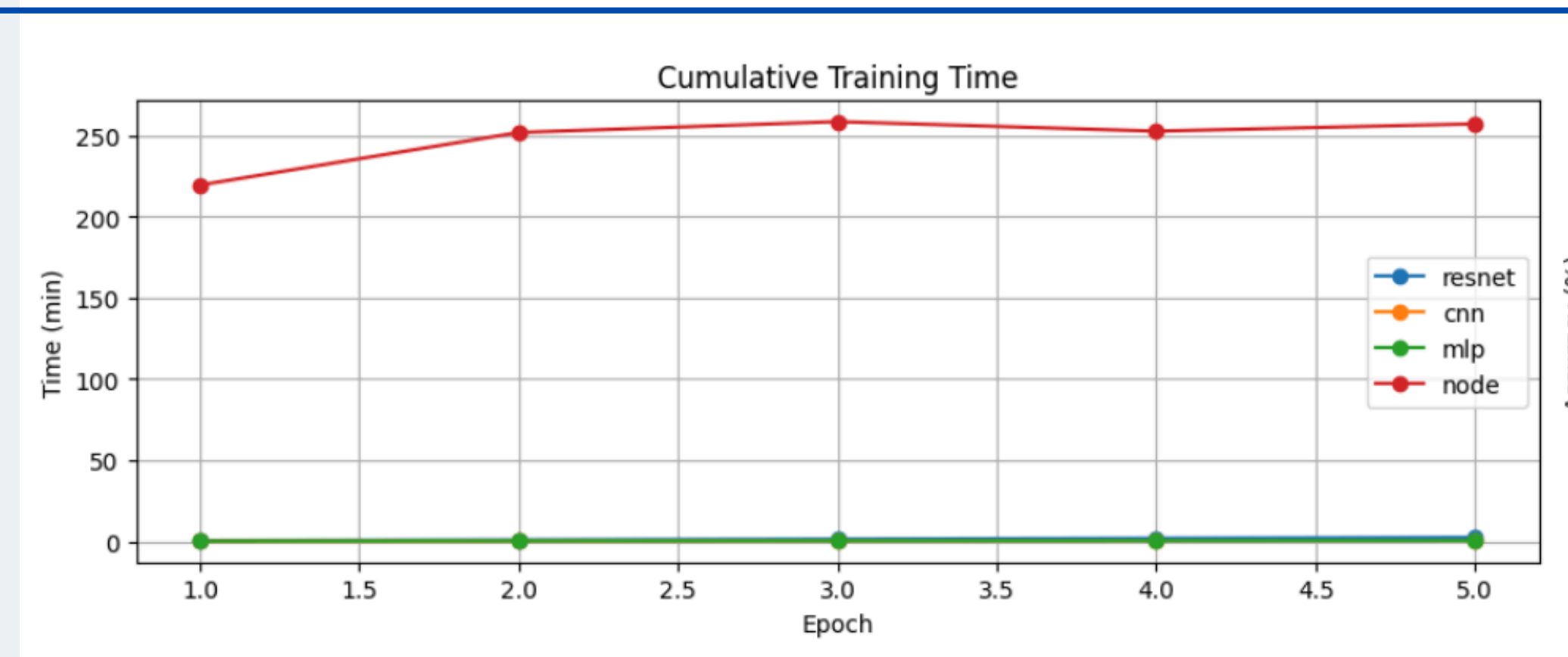
Figure 5: Comparision of Resnet, CNN, MLP, and N-ODE (using 2-order Runge-Kutta Solver) on CIFAR10 dataset

LIMITATIONS

MNIST



CIFAR10



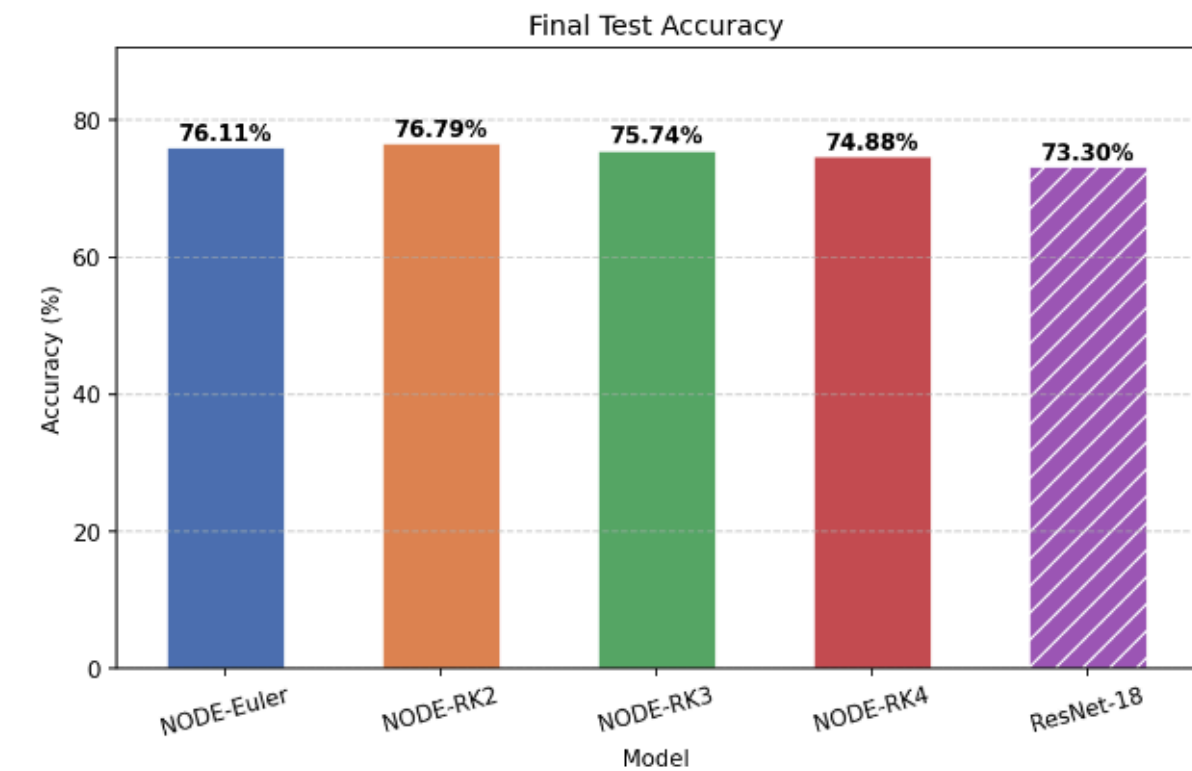
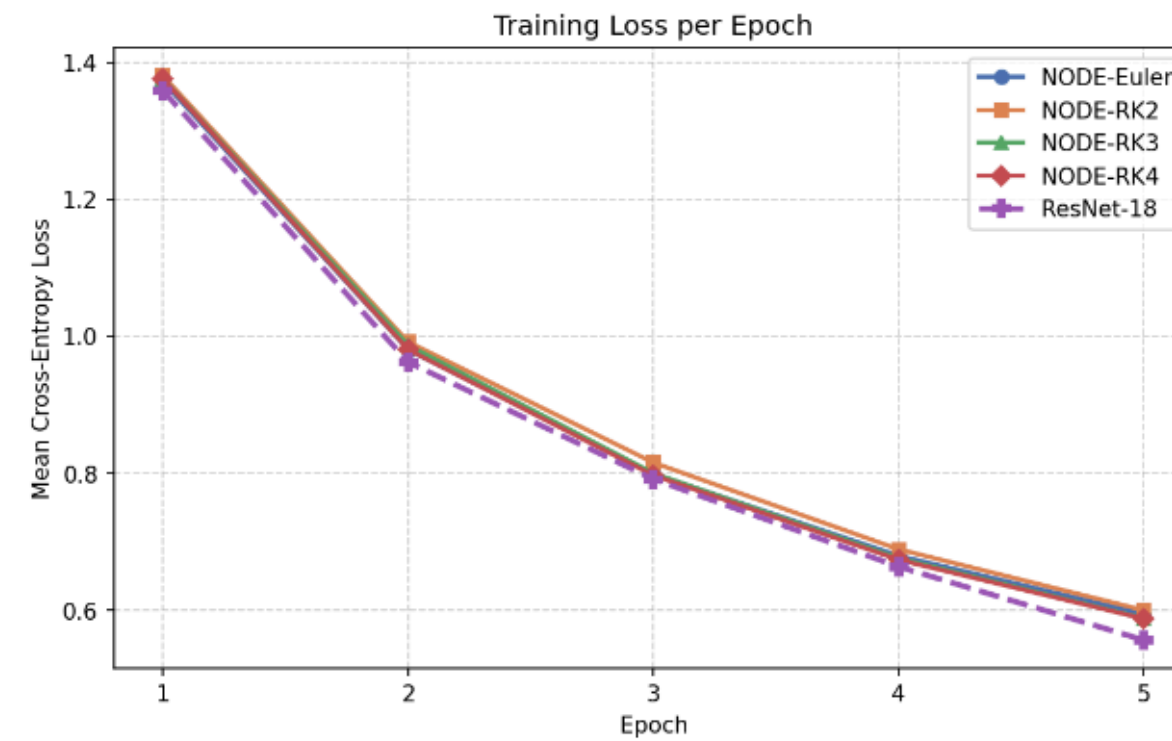
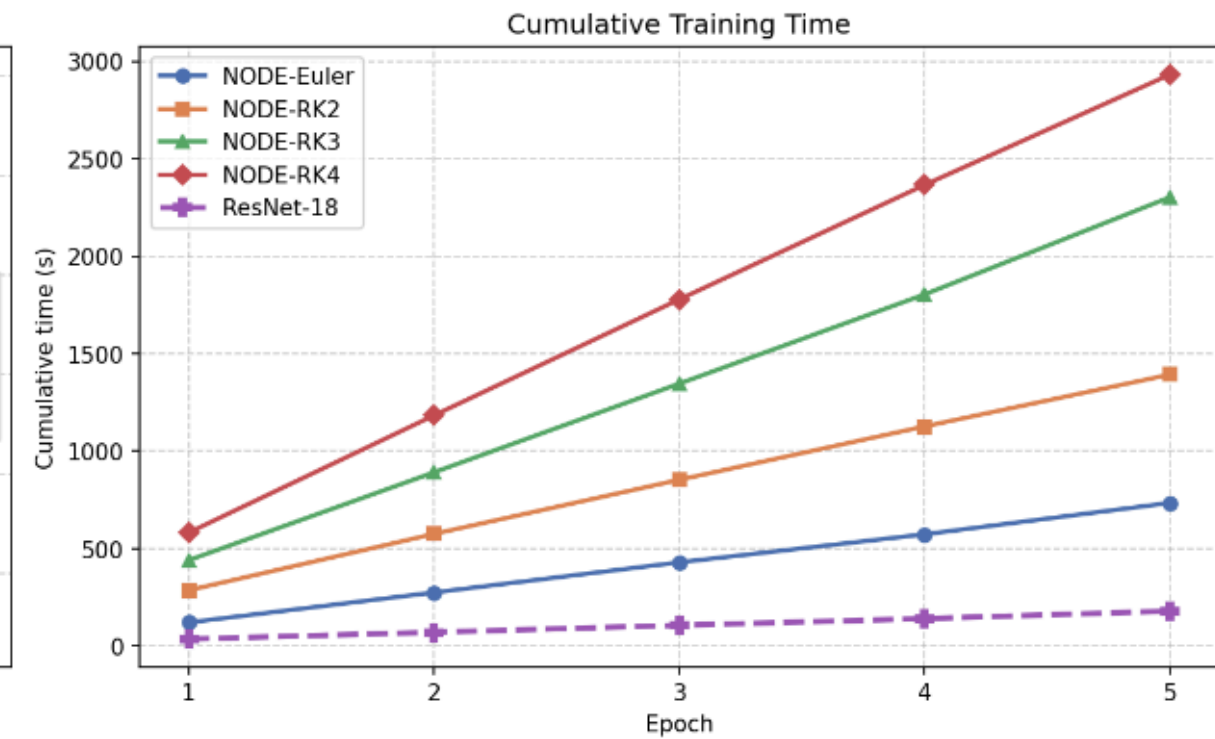
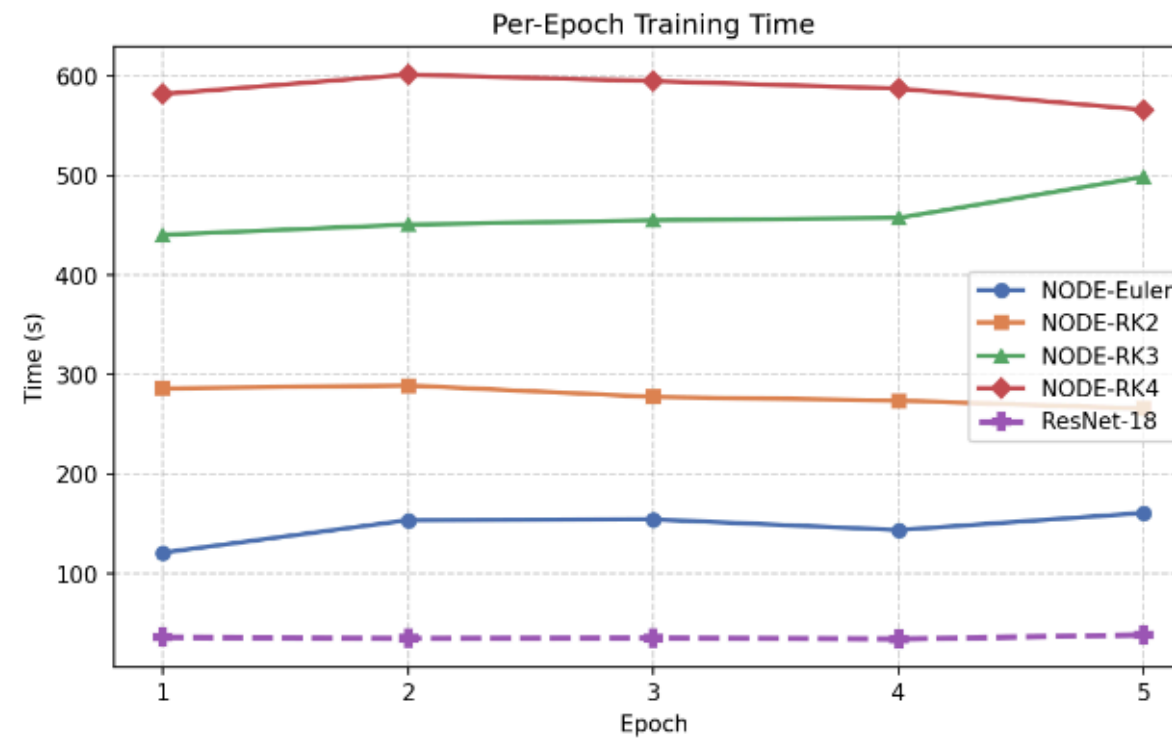
ADVANTAGES

- **Choose different solver methods**
- **Symplectic Integrators which are designed for solving Hamiltonian systems that conserve the symplectic structure of phase space.**
- **Allows for irregularly spaced data for training**
 - **when we don't have access to regularly spaced data**
 - **can be used to obtain regularly spaced data**
 - **help with interpretability of ODE**

COMPARISON OF SOLVERS

RESNET & N-ODE

Neural ODE Solvers vs ResNet-18 — CIFAR-10 Benchmark



THANK YOU!

Github repository.

Bibliography:

M. Surtsukov, Neural Ordinary Differential Equations, 2019. Available online: <https://msurtsukov.github.io/Neural-ODE/>

A. Krizhevsky, V. Nair, and G. Hinton, The CIFAR-10 and CIFAR-100 Datasets, 2009. Available online: <https://www.cs.toronto.edu/~kriz/cifar.html>

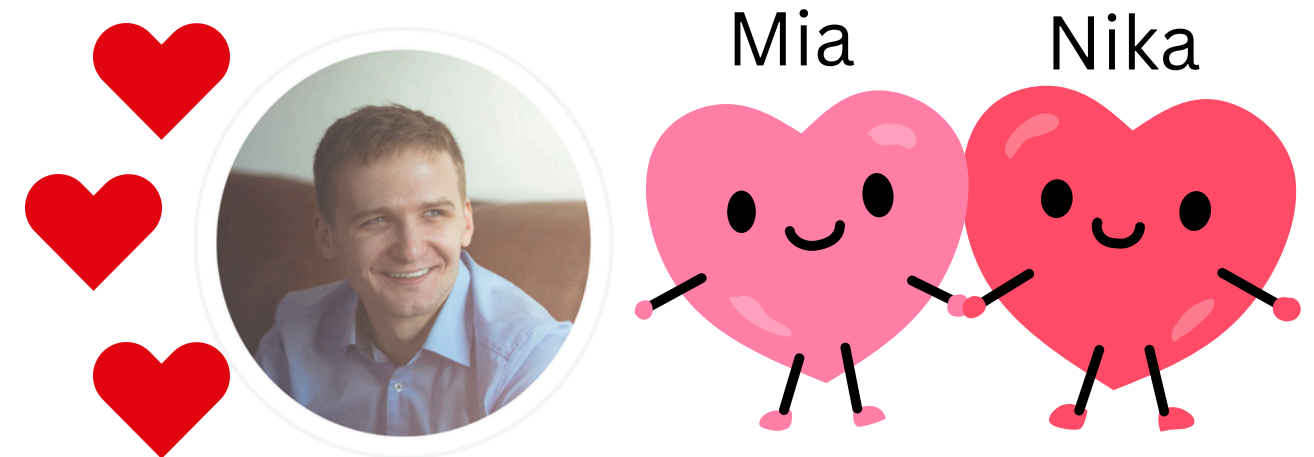
YouTube, Neural ODEs (NODEs) [Physics Informed Machine Learning]. Available online: <https://www.youtube.com/watch?v=nJphsM4obOk>

R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, Neural Ordinary Differential Equations, Advances in Neural Information Processing Systems, vol. 31, 2018. Available online: <https://papers.nips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html>

P. Kidger, On Neural Differential Equations, 2025. Available online: <https://www.sciencedirect.com/science/article/pii/S2950550X25000263>

R. Amos et al., Neural ODEs: Implicit Layers Tutorial, 2025. Available online: https://implicit-layers-tutorial.org/neural_odes/

J. Doe et al., Advances in Neural Differential Equations, Communications Biology, 2026. Available online: <https://www.nature.com/articles/s42003-026-09758-w>



Mikhail Surtsukov

I am Deep Learning enthusiast interested in
Computer Vision, Bayesian Deep Learning
and Generative Models